

VEHICLE MANAGEMENT SYSTEM

Build Your Own Project – VITyarthi

Project Report

Student Name: Harshal Pathak

Registration / Roll No.: 26BCE11365

Course / Subject: CSE(Python Programming)

1. Introduction

The Vehicle Management System is a command-line application for managing vehicles, customers and rental transactions. It uses Python for application logic and SQLite for persistent structured storage. The system is designed as an academic project that demonstrates modular programming, validation, database operations, reporting and testing.

2. Problem Statement

Small vehicle-rental or fleet operations need accurate records of vehicles, customers and rentals. Manual record keeping can make it difficult to know which vehicles are available, maintain customer records and calculate rental totals. The proposed system centralizes these operations in a simple terminal-based application.

3. Objectives

- Maintain vehicle information and availability.
- Maintain customer records.
- Start and complete rental transactions.
- Calculate rental charges automatically.
- Produce basic fleet and revenue reports.
- Demonstrate clean modular implementation and testing.

4. Functional Requirements

FR1: Add, list, search, update and delete vehicles. FR2: Add and list customers. FR3: Start rentals only for available vehicles and valid customers. FR4: Calculate estimated rental charges from daily rate and number of days. FR5: Return vehicles and update availability. FR6: Display active rentals, rental history and revenue summary.

5. Non-Functional Requirements

Performance: Use indexed primary keys and simple SQL operations suitable for a small dataset. Security: Use parameterized SQL queries to reduce SQL injection risk. Usability: Provide numbered menus, readable tables and clear error messages. Reliability: Use transactions and rollback on failed database operations. Maintainability: Separate database, validation, services, reporting and UI logic. Resource efficiency: Use SQLite and the Python standard library without external services.

6. System Architecture

The system follows a simple layered structure: CLI layer → service layer → SQLite data layer. The CLI collects user input. Service modules validate and process business rules. The database module initializes the schema and provides the connection. Reports query stored data and format summary information.

7. Design Diagrams

The repository contains the corresponding implementation. The following textual diagrams summarize the design.

Use Case Diagram

Actors: Administrator/Operator. Use cases: Manage Vehicles, Manage Customers, Start Rental, Return Vehicle, View Active Rentals, View Reports. The operator interacts with all use cases through the command-line interface.

Workflow Diagram

Start → Main Menu → Select Module → Validate Input → Execute Business Rule → Read/Write SQLite → Display Result → Return to Menu → Exit.

Sequence Diagram

Operator → CLI: enter rental details → RentalService: start_rental() → Database: validate vehicle/customer → Database: insert rental → Database: update vehicle status → RentalService → CLI: rental ID and amount.

Class / Component Diagram

main.py uses VehicleService, CustomerService, RentalService and Reports. These services use a shared SQLite connection from database.py. validators.py supplies validation functions and utils.py supplies input/table helpers.

ER Diagram

VEHICLES (vehicle_id PK, registration_no UNIQUE, vehicle_type, model, year, daily_rate, status). CUSTOMERS (customer_id PK, name, phone, license_no UNIQUE). RENTALS (rental_id PK, vehicle_id FK, customer_id FK, start_date, expected_days, return_date, total_amount, status). One vehicle can have many rental records over time; one customer can have many rental records.

8. Design Decisions & Rationale

Python standard-library tools were selected so the project can be executed from a terminal without dependency-heavy setup. SQLite was selected because it is file-based, portable and supports relational constraints. Separate service modules keep database operations and business rules organized. Transactions are used for rental start/return operations so vehicle and rental status remain consistent.

9. Implementation Details

The implementation contains multiple meaningful modules: main.py, database.py, validators.py, vehicle_service.py, customer_service.py, rental_service.py, reports.py and utils.py. The database schema is kept separately in schema.sql. The tests directory contains unit/validation tests. The command line is the only interface, satisfying the terminal execution requirement.

10. Screenshots / Results

For the final submission, run the application and capture screenshots of: main menu, vehicle list, customer list, successful rental, vehicle return and reports. These screenshots should be added to the repository or inserted into the final PDF after personal testing.

11. Testing Approach

Unit tests use an in-memory SQLite database so tests do not modify the project database. Tests cover vehicle/customer creation, rental charge calculation, vehicle status changes after rental and return, and invalid phone input. Run: `python -m unittest discover -s tests -v`

12. Challenges Faced

Important implementation challenges include keeping vehicle availability synchronized with rental records, validating user input, handling duplicate database values and keeping the command-line interface separate from business logic.

13. Learnings & Key Takeaways

The project demonstrates practical use of Python classes, functions, exception handling, SQL CRUD operations, relational constraints, modular design, automated testing and version-control-ready project organization.

14. Future Enhancements

Possible extensions include login/role management, maintenance scheduling, late-return charges, PDF/CSV reports, web or GUI interface, payment integration, vehicle service reminders and analytics dashboards.

15. References

1. VITyarthi Build Your Own Project – General Project Instructions & Submission Guidelines (provided course document). 2. Python 3 documentation. 3. SQLite documentation. 4. Python unittest documentation.